

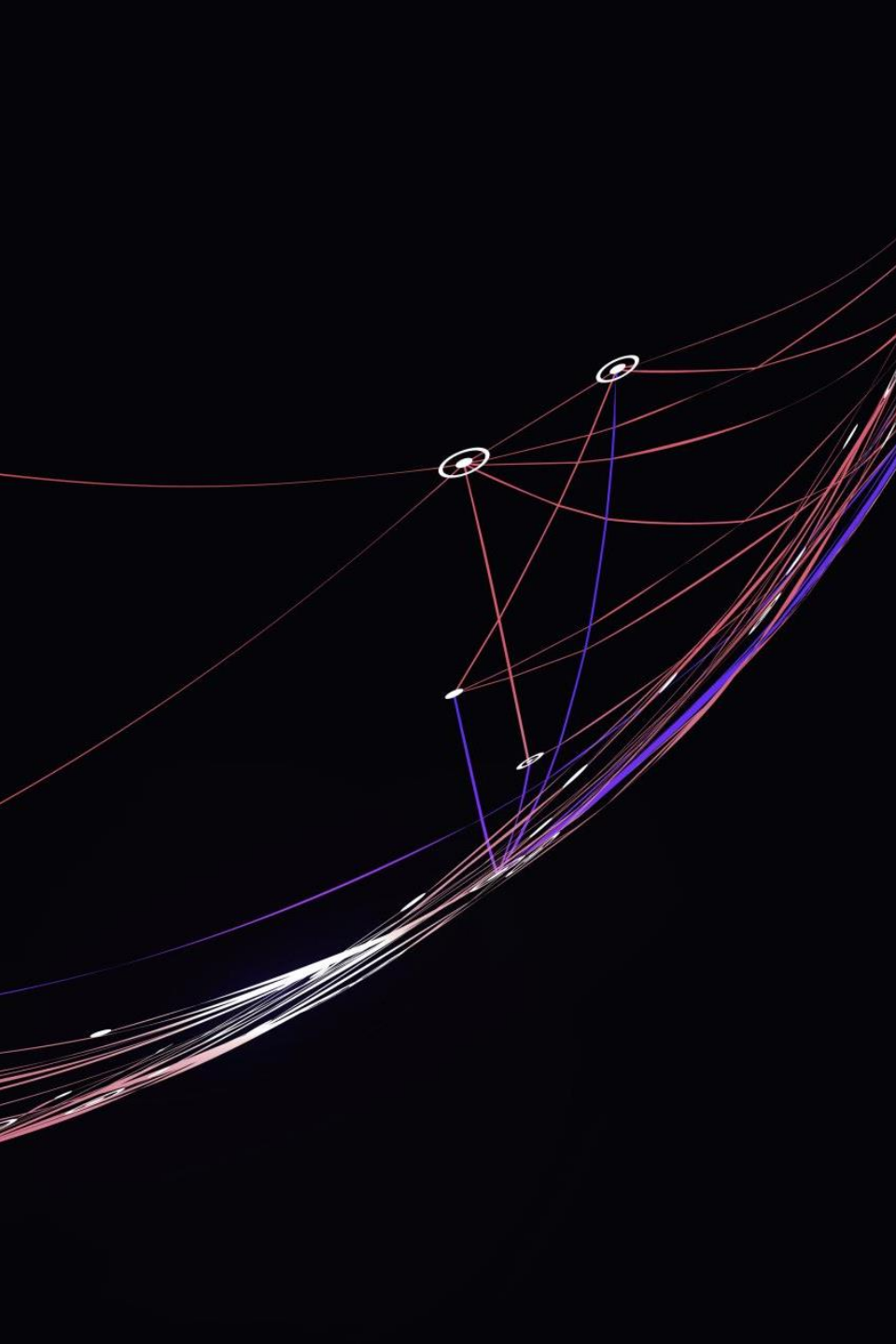
Integration of a Reinforcement Learning-Based Neural Network into the Crazyflie Flight Controller

ANDREA ARGNANI

MASSIMILIANO BILLI

FEDERICO MAURI

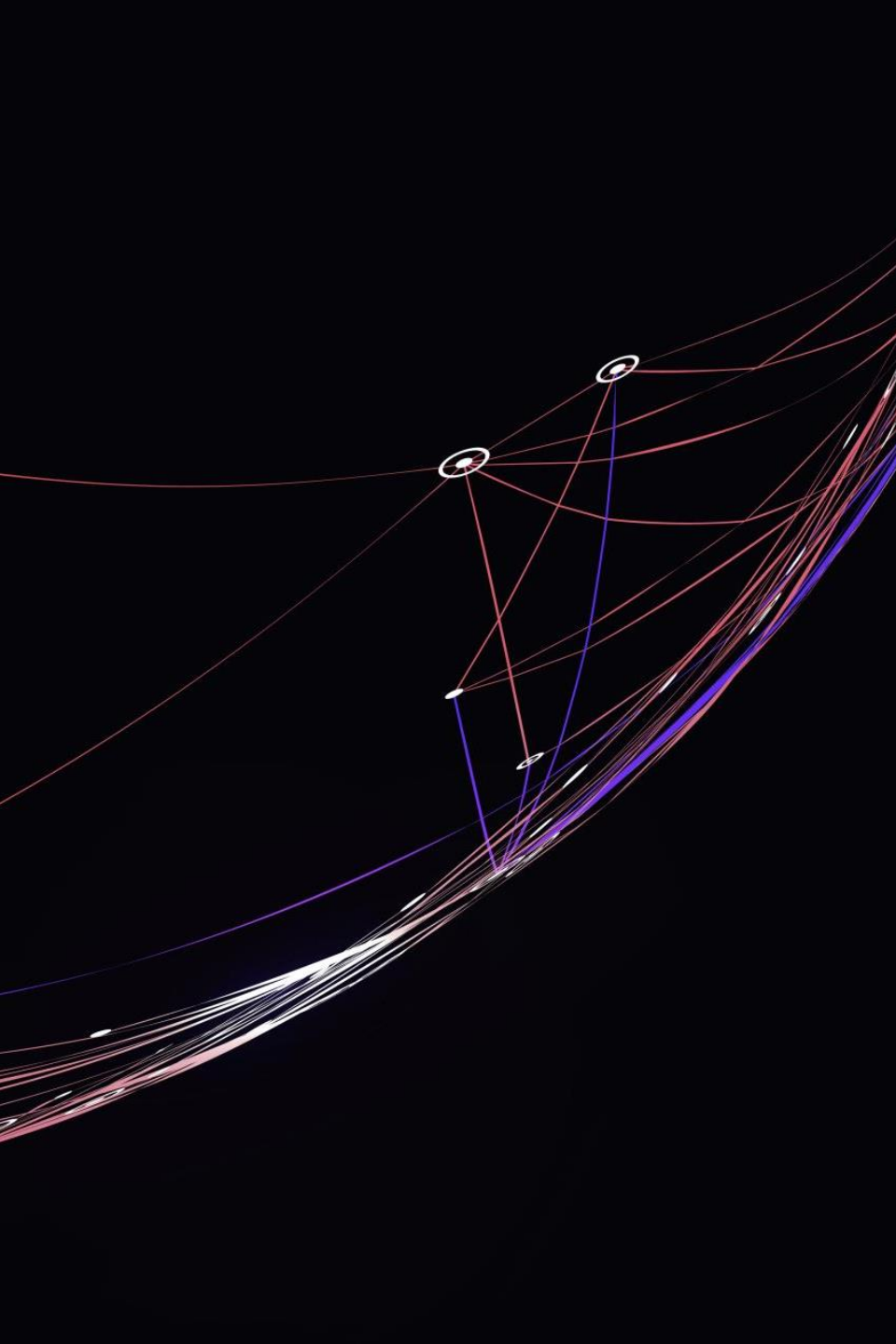


An abstract graphic on the left side of the slide, featuring a dark background with a complex network of glowing lines in red, blue, and white. These lines represent connections between nodes, with some nodes highlighted as small circles. The lines curve and flow from the bottom left towards the top right, creating a sense of dynamic movement and connectivity.

Objective:

The goal of this project is to integrate an Artificial Neural Network (ANN), trained through Reinforcement Learning, into the Crazyflie flight platform to solve a hovering control task.

Interface the ANN with sensors and control stack. Evaluate execution time and memory loads.



Repository structure:

- ❖ *src/ANN_crazyflie.c*

Contains the C implementation of the ANN.

- ❖ *get_header.py*

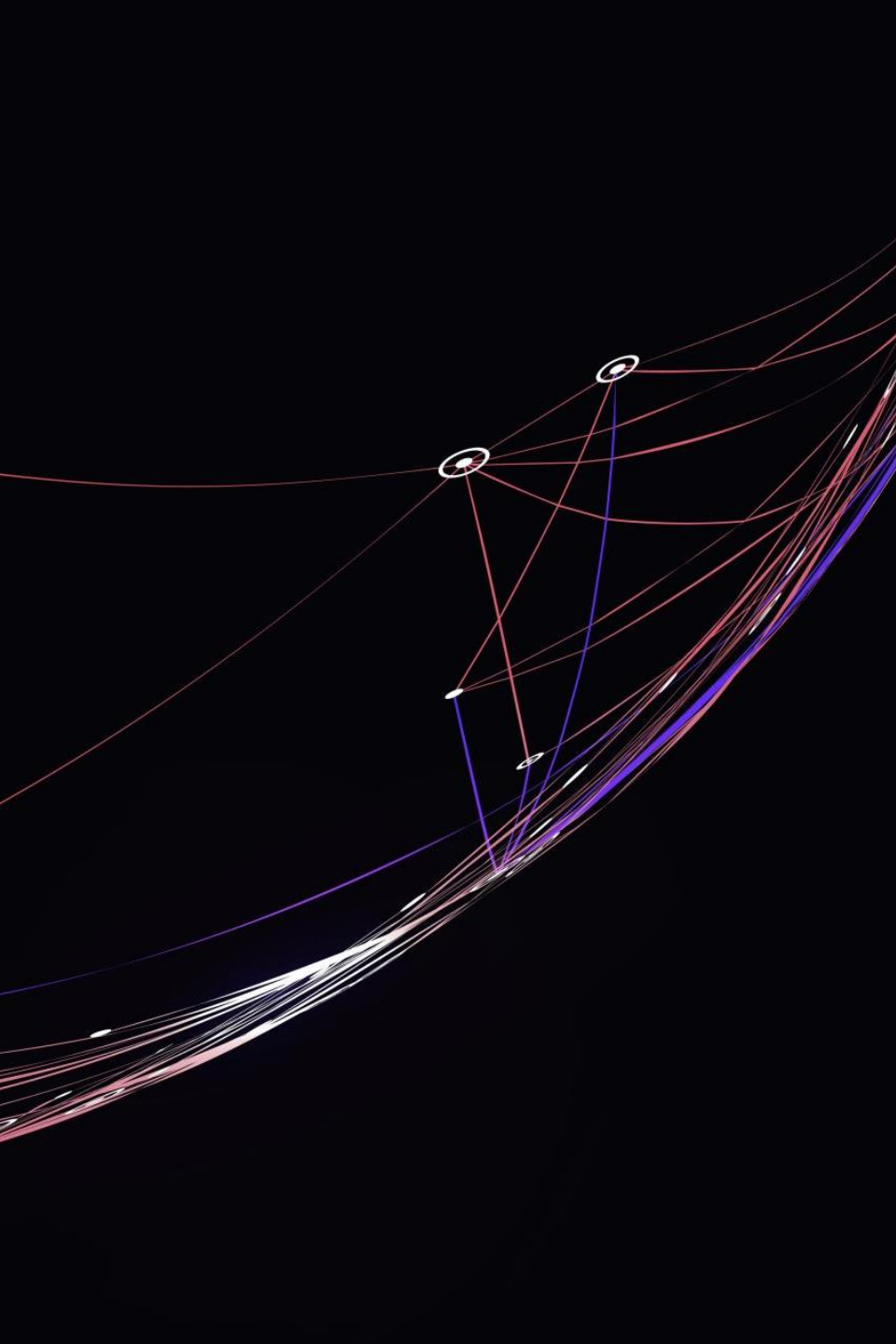
Python script used to convert the model's weights, written in `best_agent_noise01.pt` in PyTorch format, into a C-compatible header file (`src/pesi_modello.h`).

- ❖ *golden_model.py*

Python implementation of the same neural network used as a reference (golden model) to validate the C version's behavior of `ANN_crazyflie.c`.

- ❖ *get_header_random.py*

Python script to generate random weights and layers to perform time analysis with different model sizes.



ANN implementation:

appMain structure:

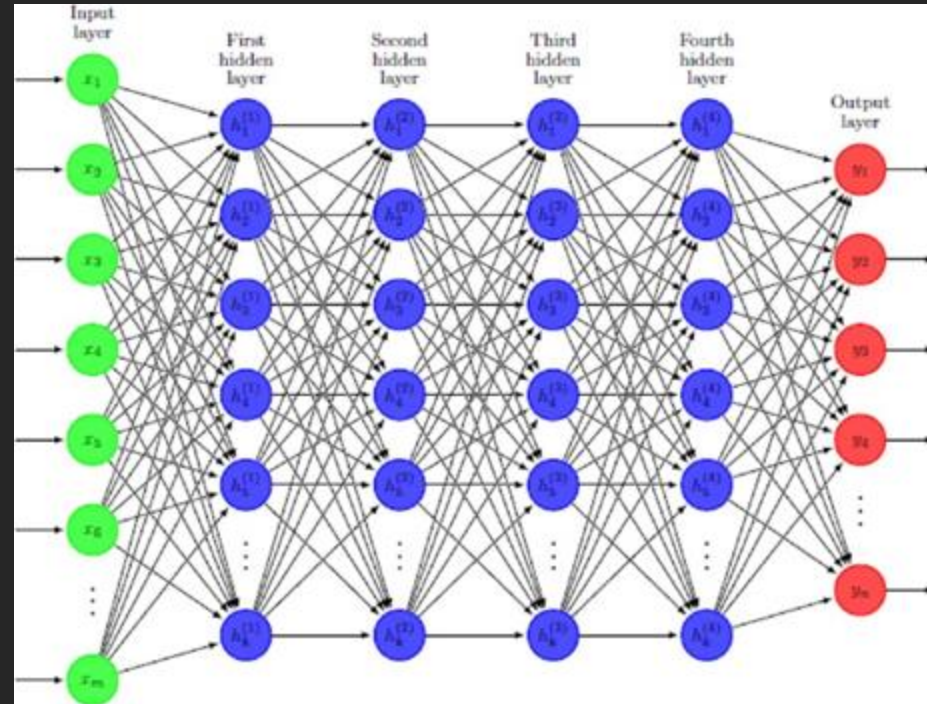
- ❖ Sensors reading
- ❖ Artificial Neural Network execution
- ❖ Motors control
 - ❖ Via Setpoint
 - ❖ Direct control via PWM

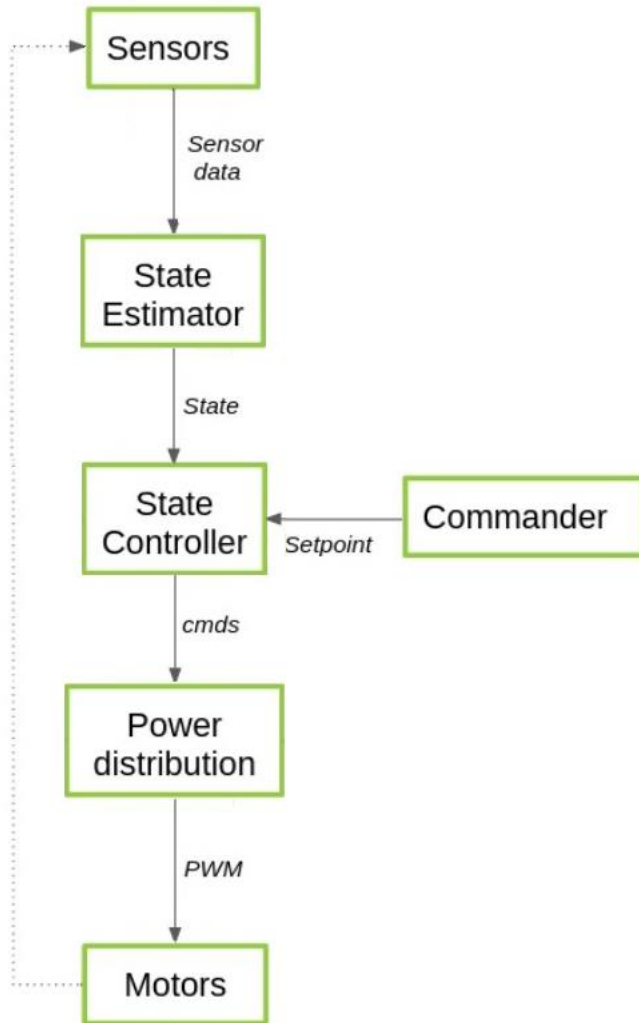
Artificial Neural Network

We used the ARM CMSIS-DSP library optimized for Cortex-M processors, to perform:

- ❖ Matrix-vector multiplications
- ❖ Vector additions

```
for( int l = 0 ; l < N_LAYER ; l++)  
{  
    int n = net_weights[l].numRows;  
    arm_mat_vec_mult_f32( &net_weights[l], buffer_1, buffer_2);  
    arm_add_f32( buffer_2, net_bias[l], buffer_1, n );  
    // attivazione  
    if( l < N_LAYER-1)  
        relu_f32 ( buffer_1 , n );  
}  
  
for(int i=0; i<4; i++)  
    output[i] = buffer_1[i];
```

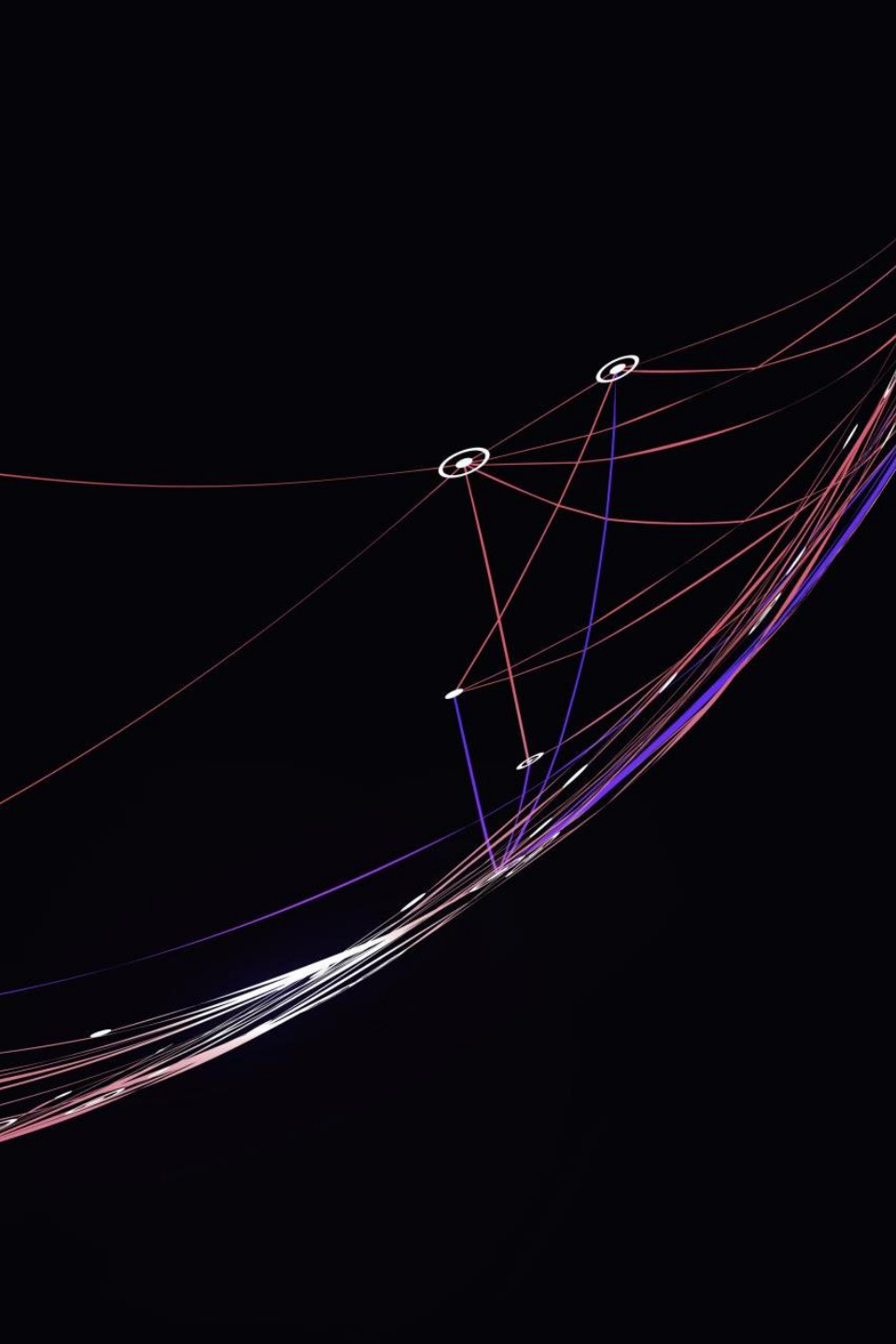




Out-of-tree controller:

Used to run with higher execution priority

- ❖ Initialization with the *controllerOutOfTreeInit()*
- ❖ Tested using the golden model
- ❖ Realization of *controllerOutOfTree()*
 - ❖ Sensors reading
 - ❖ ANN execution
 - ❖ Direct motors override

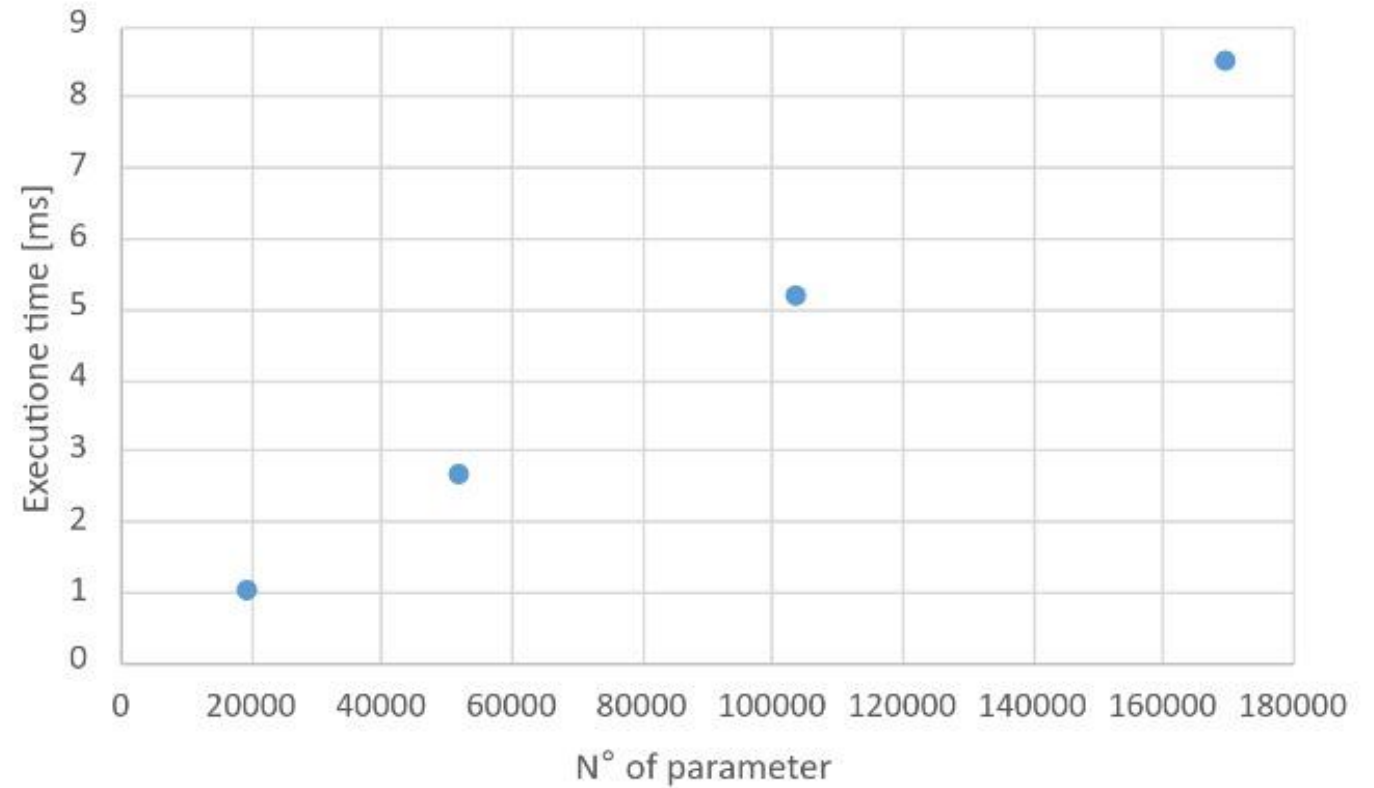
An abstract graphic on the left side of the slide, featuring a dark background with a series of glowing, curved lines in shades of red, orange, and blue. These lines represent connections between nodes, with some nodes highlighted as small white circles with black outlines. The lines flow from the bottom left towards the top right, creating a sense of dynamic movement and complexity.

Memory occupation:

The artificial neural network (ANN) occupies approximately 400 KB, resulting in a total firmware size of 677 KB. Given that the Crazyflie's RAM is limited to only 196 KB, we decided to store the ANN weights in the FLASH memory (1 MB). To achieve this, we declared the weights using the *const* keyword.

Time analysis:

We generated multiple models with varying numbers of layers and total parameters using the *get_header_random.py* script, in order to evaluate the trade-off between the ANN's complexity and its execution time.



Future developments:

- ❖ Ensure that the Crazyradio link operates correctly when an out-of-tree controller is in use.
- ❖ Optimize the execution of the ANN via DMA
- ❖ Implementation on the AI-Deck



Thanks for the attention
